

DOSSIER DE PROJET

Titre Professionnel Développeur Web et Web Mobile – Niveau 5

notSafe

Plateforme d'audit de mots de passe et de surveillance des fuites de données

Candidat	Igor Luna de Oliveira
Formation	DWWM – Développeur Web et Web Mobile
Centre	IMIE - Paris
Session	Mai 2026

Remerciements

Je tiens à remercier l'équipe pédagogique de l'IMIE pour leur soutien tout au long de cette formation. Leurs conseils m'ont poussé à comprendre le fonctionnement réel des outils plutôt que de simplement les utiliser sans en saisir la logique.

Merci également à mes camarades de promotion. Et à ma famille qui m'a soutenu pendant la formation

Ce projet représente pour moi l'aboutissement de mois de travail et d'une passion grandissante pour le développement web.

Sommaire

1. Présentation du projet et contexte
2. Gestion de projet — Méthode Agile
3. Cahiers des Charges
4. Conception — Maquettes
5. Architecture de données
6. Développement Backend — API Flask
7. Développement Frontend — React
8. Sécurité
9. Déploiement — Docker
10. Bilan des compétences DWWM
11. Conclusion personnelle

Annexes

1. Présentation du projet et contexte

1.1 Mon parcours

Je m'appelle Igor Luna de Oliveira. J'ai intégré la formation DWWM à l'IMIE avec un objectif précis : devenir développeur web en me spécialisant sur les aspects sécurité. Pendant ma formation, j'ai appris à concevoir des interfaces, à créer des API, à modéliser des bases de données et à déployer des applications. Ce dossier présente le projet que j'ai choisi pour mettre toutes ces compétences en pratique.

1.2 L'idée de notSafe

L'idée de notSafe est venue pendant un cours où l'on a étudié RockYou.txt, un fichier de 150 Go qui contient des milliards de mots de passe.

Au départ, je voulais simplement faire une page pour vérifier si mon propre mot de passe était dans la liste. Mais j'ai vite réalisé qu'en tant que junior, lire un fichier de 150 Go sur un PC classique était impossible. J'étais bloqué par la puissance de ma machine.

Pour avancer, j'ai d'abord construit un prototype avec une version de 100 Mo du fichier, puis j'ai intégré l'API de *Have I Been Pwned* pour que l'outil soit vraiment efficace sans faire ramer l'ordinateur.

Comme je ne pouvais pas présenter une simple barre de recherche pour mon projet final, j'ai décidé de transformer l'essai. J'ai supprimé le fichier texte local pour construire une plateforme complète, sur le modèle d'un SaaS :

- Un tableau de bord pour suivre l'évolution de la sécurité.
- Une page dédiée aux entreprises pour gérer les accès.
- Un système de vérification d'emails pour détecter les fuites de données.

Le projet est passé d'un simple test technique à un outil visuel et pratique pour aider les structures qui n'ont pas forcément les moyens de se payer des logiciels de sécurité complexes.

1.3 Stack technique choisie

Pour ce projet, j'ai sélectionné des outils en cohérence avec ma formation DWWM, en privilégiant des technologies que je maîtrisais déjà tout en intégrant des langages dans lesquels je voulais monter en compétence.

- Frontend : React 18 + Vite – Mon choix pour créer une interface (SPA) fluide et moderne. Vite me permet d'avoir un environnement de développement très réactif.
- Backend : Flask (Python) – J'ai choisi Flask par intérêt personnel et pour sortir de ma zone de confort. C'est un framework léger qui me permet de progresser sur Python tout en construisant une API REST robuste.
- Bases de données :
 - SQL via SQLAlchemy : À l'origine, je pensais partir sur du 100% NoSQL. Cependant, pour valider mes compétences en base de données relationnelle, j'ai intégré SQLAlchemy. Cela m'a permis de gérer proprement mes données structurées tout en montrant ma maîtrise du SQL.
 - NoSQL (MongoDB) : Je l'ai conservé pour le stockage des logs et des statistiques anonymes, car la flexibilité du NoSQL reste la plus adaptée pour ce type de données non-structurées.
- Déploiement : Docker – Utilisation de Docker et Docker-compose pour garantir que le projet fonctionne exactement de la même manière sur mon poste et en production, évitant les problèmes de compatibilité.

2. Gestion de projet — Méthode Agile (Scrum)

2.1 Méthodologie Agile : Le choix de l'efficacité

Pour le développement de notSafe, j'ai choisi la méthodologie Scrum. En tant que développeur seul sur le projet, cela m'a permis de rester organisé et de ne pas me perdre face à l'ampleur des fonctionnalités à livrer.

J'ai structuré mon travail autour d'un Sprint de 2 semaines. L'idée n'était pas de tout coder d'un coup, mais de livrer des fonctionnalités testables rapidement. À la fin de chaque Sprint, j'avais quelque chose qui fonctionnait — même si c'était incomplet. Ça m'a aidé à rester motivé et à corriger le tir au bon moment plutôt qu'en toute fin de projet.

2.2 Organisation des Sprints

Le projet s'est déroulé sur trois mois, du 2 février au 1er mai 2026, découpé en sept Sprints de deux semaines chacun.

Le **Sprint 0 (02/02 – 15/02)** a été consacré à la conception. Avant d'écrire la moindre ligne de code, j'ai modélisé la base de données avec la méthode Merise, dessiné les maquettes sur Figma et choisi la stack technique définitive. C'est le Sprint le plus ingrat mais le plus important : tout ce qui vient après en dépend.

Le **Sprint 1 (16/02 – 01/03)** a posé le socle applicatif. J'ai créé les modèles SQLAlchemy, mis en place l'inscription et la connexion sécurisées avec Bcrypt et JWT, et implémenté le décorateur `@token_required` qui protège toutes les routes sensibles.

Le **Sprint 2 (02/03 – 15/03)** a été le plus dense techniquement. J'y ai développé le cœur du backend : le proxy HIBP avec le protocole k-anonymité, le scan email via XposedOrNot, le flow complet de réinitialisation de mot de passe, et la journalisation anonyme dans MongoDB.

Le **Sprint 3 (16/03 – 29/03)** a marqué le passage au frontend. J'ai construit toutes les pages React publiques — le Password Auditor, l'Email Checker, le portail employé — ainsi que l'instance Axios centralisée avec ses intercepteurs.

Le **Sprint 4 (30/03 – 12/04)** a été dédié au dashboard administrateur : les graphiques Chart.js, la gestion des départements, et les pipelines d'agrégation MongoDB pour les statistiques en temps réel.

Le **Sprint 5 (13/04 – 26/04)** a servi à stabiliser l'ensemble. J'ai écrit les tests, conteneurisé l'application avec Docker, corrigé les incohérences repérées en relisant le code, et rédigé le README.

Le **Sprint 6 (27/04 – 01/05)** a été entièrement consacré à la soutenance : rédaction du dossier, préparation du diaporama, et répétition de la présentation orale.

2.3 Outils utilisés

- Git & GitHub — Versionnage du code avec une branche par fonctionnalité. Commits réguliers pour ne rien perdre.
- Figma — Conception des maquettes avant de toucher au code.
- Docker — Conteneurisation de l'application dès le Sprint 5. Plutôt que de livrer un projet qui « marche sur ma machine », Docker m'a permis d'emballer le backend, le frontend et les deux bases de données dans des conteneurs standardisés.

Release Planning — notSafe

Projet du 02/02/2026 au 01/05/2026 – 7 Sprints de 2 semaines

Légende : **Sprint 0** = Conception **Sprint 1–2** = Backend **Sprint 3–4** = Frontend **Sprint 5–6** = Finalisation

Sprint 0

02/02 – 15/02

Objectif :
Conception &
Setup

- Initialisation du dépôt Git et structure des dossiers (routes/, models/, services/)
- Configuration de l'environnement virtuel Flask, installation des dépendances (requirements.txt)
- Modélisation Merise : MCD → MLD → MPD (entités Company, User, Department)
- Conception des maquettes sur Figma : zoning, wireframes, haute fidélité
- Choix de la stack technique finale et rédaction du cahier des charges
- Setup de la base SQLite de développement avec Flask-Migrate

Sprint 1

16/02 – 01/03

Objectif :
Socle &
Authentification

- Création des modèles SQLAlchemy : Company, User, Department avec UUIDs
- Endpoint POST /api/v1/auth/register : création entreprise + admin + 10 départements par défaut
- Endpoint POST /api/v1/auth/login : vérification Bcrypt, génération JWT 24h
- Décorateur @token_required : middleware de protection des routes
- Politique « zéro stockage » : aucun mot de passe jamais persisté en clair
- Tests Postman : register → login → accès protégé

Sprint 2

02/03 – 15/03

Objectif :
Cœur métier
Backend

- Proxy HIBP k-anonymité : GET /check-prefix/{5chars} — validation hex stricte, cache 5 min
- Calcul d'entropie Shannon côté client (utils.js) : $E = L \times \log_2(R)$, estimation temps de crack
- Scan email XposedOrNot : POST /email-check — rate limiting 10 req/min, score de risque
- Flow reset mot de passe : forgot-password (anti-énumération) + reset-password (token JWT 1h, jti)
- Logs anonymes MongoDB : collection checks (type, length, is_breached, timestamp)
- Endpoints publics : /public/stats et /public/company/{id}

Sprint 3

16/03 – 29/03

Objectif :
Frontend React —
Outils publics

- Setup Vite + React 18 + React Router v7 + Bootstrap 5 + glassmorphism CSS
- Instance Axios centralisée (api.js) : intercepteur JWT auto-inject + gestion 401
- Hook usePasswordHasher : SHA-1 via Web Crypto API, séparation prefix/suffix
- Page PasswordTool : entropie temps réel, badge de force, vérification HIBP
- Page EmailChecker : formulaire de scan, affichage des sources de fuite
- Page CompanyPortal : portail employé par entreprise (/portal/:companyId)
- Pages Login, Register, ForgotPassword, ResetPassword avec validation live
- PrivateRoute + Footer + Home

Sprint 4

30/03 – 12/04

Objectif :
Dashboard &
Analytics

- Dashboard admin protégé : métriques (total checks, breaches, safety score)
- Graphique Bar Chart.js par département (total checks vs breaches)
- Gestion des départements : ajout avec validation XSS, suppression, lien portail
- GlobalDashboard public : Line chart 30 jours, Doughnut ratio email, distribution longueurs
- Agrégation MongoDB : pipeline par département, tendances temporelles

– Endpoint GET /dashboard/analytics et GET /dashboard/global

Sprint 5

13/04 – 26/04

Objectif :
Tests, Docker &
Stabilisation

- Tests unitaires (test_unit_pure.py) : 32 tests sur fonctions pures — sanitize_name, prefix_hex, email
- Tests d'intégration (test_integration_endpoints.py) : Flask test client + SQLite :memory:
- Tests fonctionnels (test_functional_journeys.py) : parcours complets end-to-end
- Tests sécurité (test_security.py) : JWT forgé, alg:none, XSS payloads, isolation entreprises
- Conteneurisation Docker : Dockerfiles backend + frontend, docker-compose multi-services
- Correction Register.jsx (fetch → api.js), suppression lien dev dans Login.jsx
- Rédaction README.md avec instructions d'installation et .env.example

Sprint 6

27/04 – 01/05

Objectif :
Livraison DWWM

- Rédaction du dossier de projet (Cahier des charges, architecture, sécurité, bilan compétences)
- Préparation du diaporama de présentation (18 slides)
- Revue finale du code : cohérence, commentaires, nettoyage
- Mise en ligne de la démo (hébergement de présentation)
- Répétition de la soutenance orale — anticipation des questions jury

3. Cahier des charges

3.1 Contexte et objectif

notSafe est né du constat que les PME sont les premières victimes des fuites de données massives (plus de 8 milliards de comptes compromis en 2024) sans avoir les outils pour se protéger. L'objectif est de fournir une solution d'audit **Full-Stack** permettant de passer d'une sécurité subie à une culture de la prévention, le tout sans jamais stocker de données sensibles.

3.2 Fonctionnalités attendues

Le périmètre fonctionnel a été priorisé selon la méthode **MoSCoW** :

- **Outils Publics (Must Have) :** Analyseur d'entropie (calcul du temps de piratage théorique), scanner de fuites d'emails (via l'API HIBP) et dashboard de santé globale.
- **Gestion Entreprise (Must Have) :** Back-office sécurisé par **JWT** permettant la gestion granulaire des départements et la visualisation des risques via des graphiques de monitoring.
- **Portail Employé (Should Have) :** Interface simplifiée permettant un audit anonymisé, où chaque test alimente les statistiques de l'entreprise via un système de journalisation sans persistance des saisies.

3.3 Contraintes techniques

L'application repose sur une architecture moderne et découplée :

- **Frontend :** SPA développée avec **React 18** et **Vite**, garantissant une interface réactive et performante.
- **Backend :** API REST sous **Flask**, organisée en **Blueprints** pour faciliter la maintenance et l'évolution du code.
- **Persistance Hybride :** Utilisation de **PostgreSQL** (via SQLAlchemy) avec des identifiants **UUID v4** pour la sécurité des données métier, et **MongoDB** pour la flexibilité des logs analytiques.
- **Infrastructure :** Conteneurisation avec **Docker** et **Docker-compose** pour assurer l'isomorphisme entre les environnements de développement et de production.

3.4 Exigences de sécurité

Le projet suit les principes du "Security by Design" :

1. **Confidentialité absolue :** Implémentation du protocole de **k-anonymité** (hachage SHA-1 partiel) pour interroger les bases de fuites externes sans jamais transmettre le mot de passe complet.
2. **Zéro stockage sensible :** Aucune saisie utilisateur n'est stockée. Seuls les comptes administrateurs sont persistés avec un hachage **Bcrypt** à haut facteur de travail.

3. **Protection des accès** : Authentification par token **JWT** avec expiration et mise en place de **Rate Limiting** pour prévenir les attaques par déni de service ou brute-force.
4. **Intégrité du code** : Sanitisation systématique des entrées pour bloquer les failles XSS et les injections SQL, validée par des critères de succès stricts dans la **Definition of Done (DoD)**.

4. Conception — Maquettes et architecture de données

4.1 Approche et Philosophie

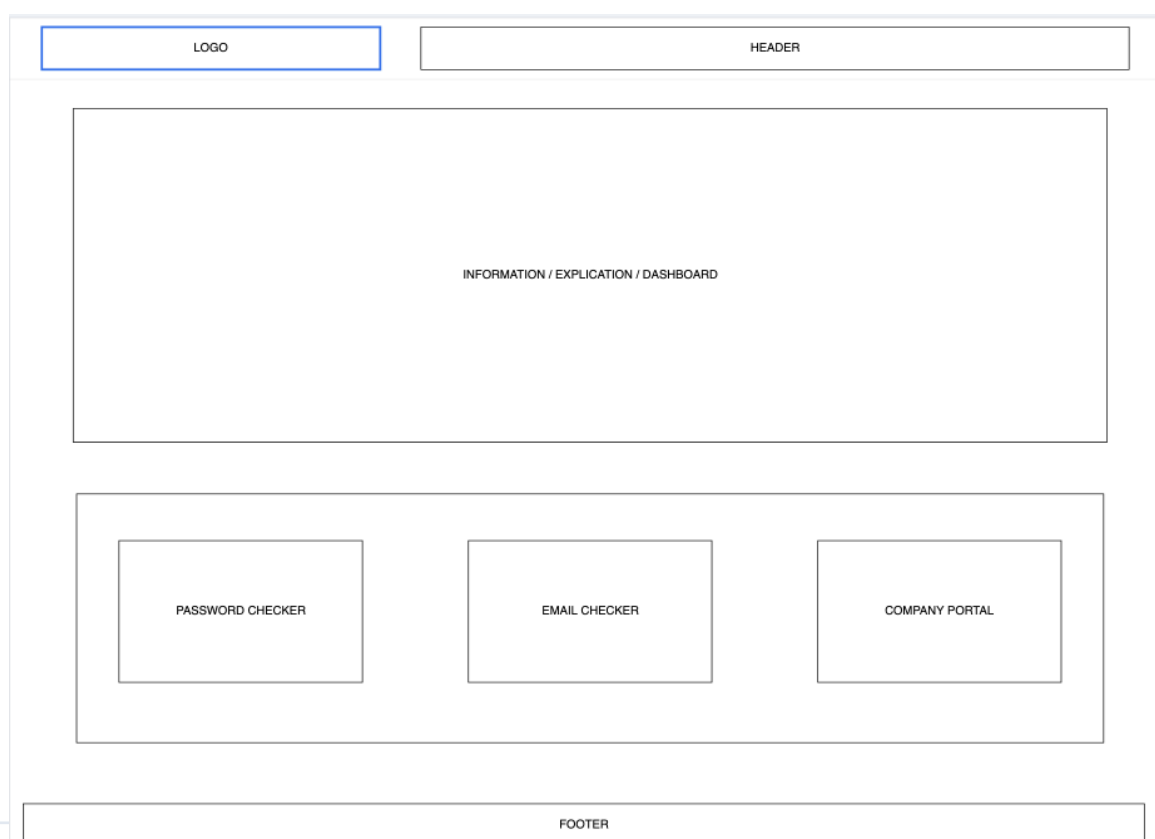
L'objectif de **notSafe** est de rendre la cybersécurité accessible. J'ai donc privilégié une interface "Clean & Safe" : des couleurs qui inspirent la confiance (bleu foncé, gris acier) et une navigation simplifiée pour que l'utilisateur ne se sente pas submergé par la complexité technique.

4.2 La phase de conception (Zoning & Wireframes)

Pour construire l'interface, j'ai suivi une méthodologie en trois étapes

- **Le Zoning (Structure)** : J'ai d'abord défini les zones de contenu sans me soucier du design pour une lecture rapide.

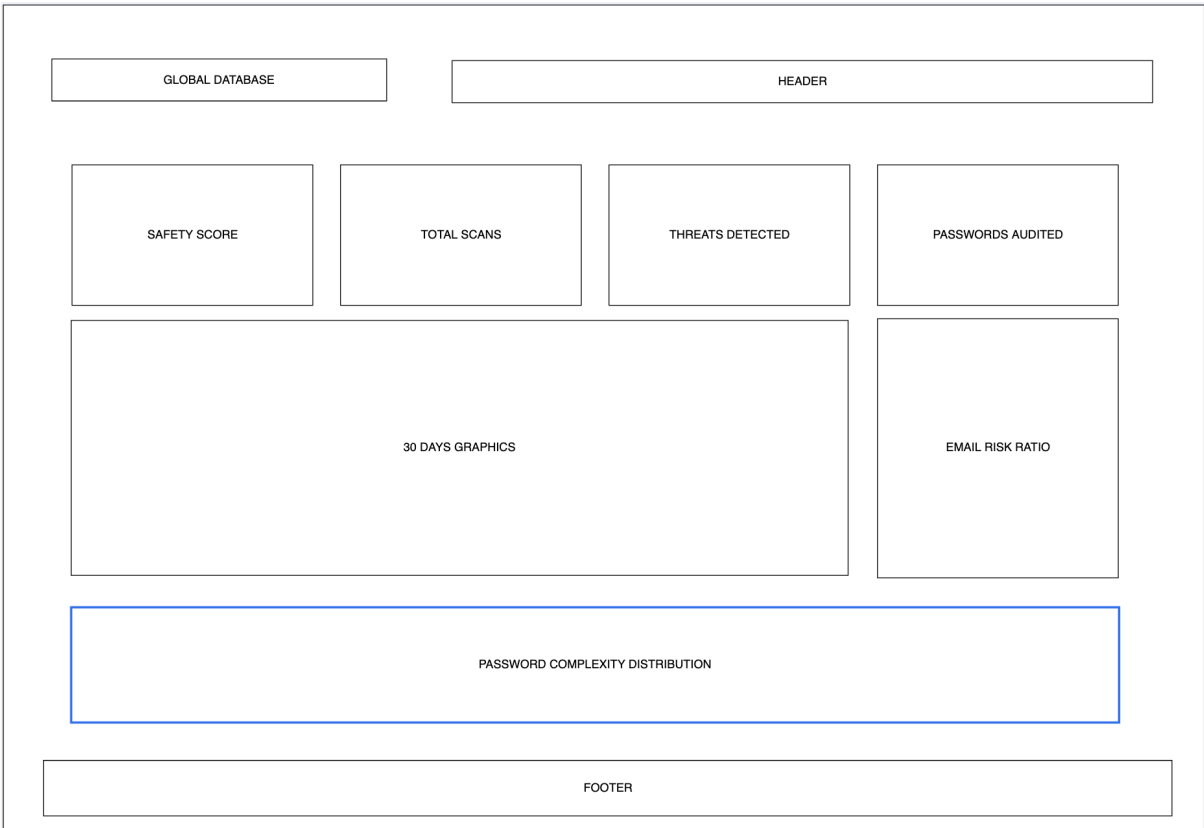
Zoning de la Home Page - notSafe.



Zoning de l'audit de mot de passe - notSafe.

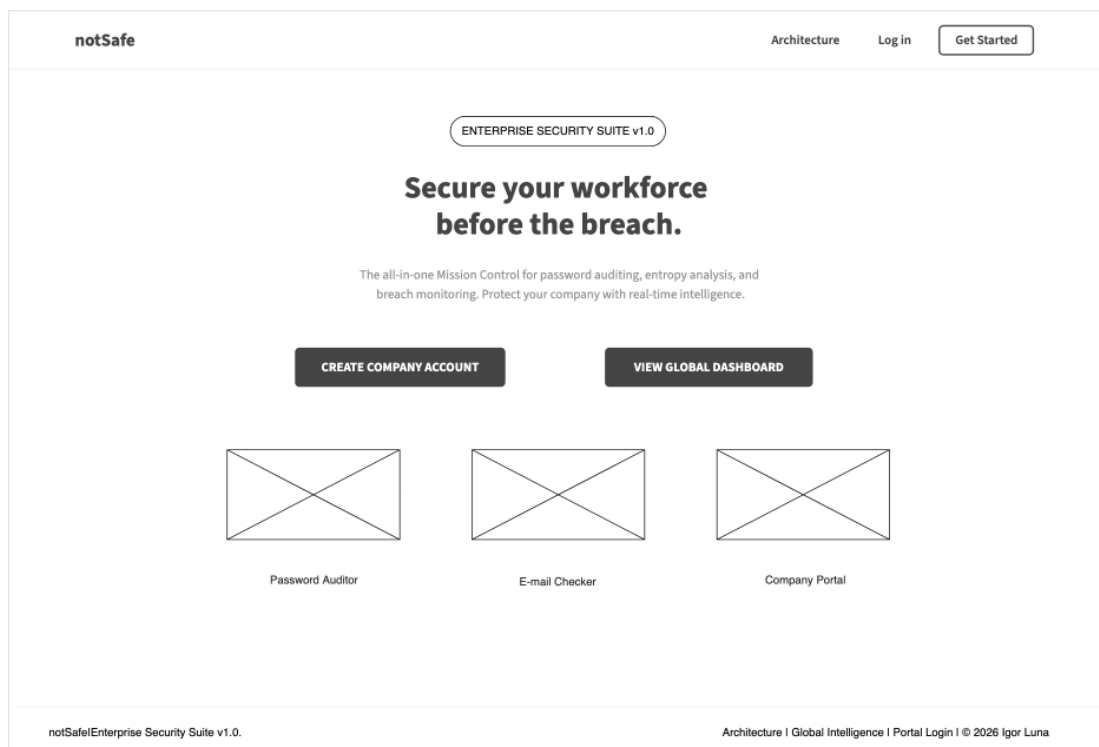


Zoning du dashboard - notSafe.

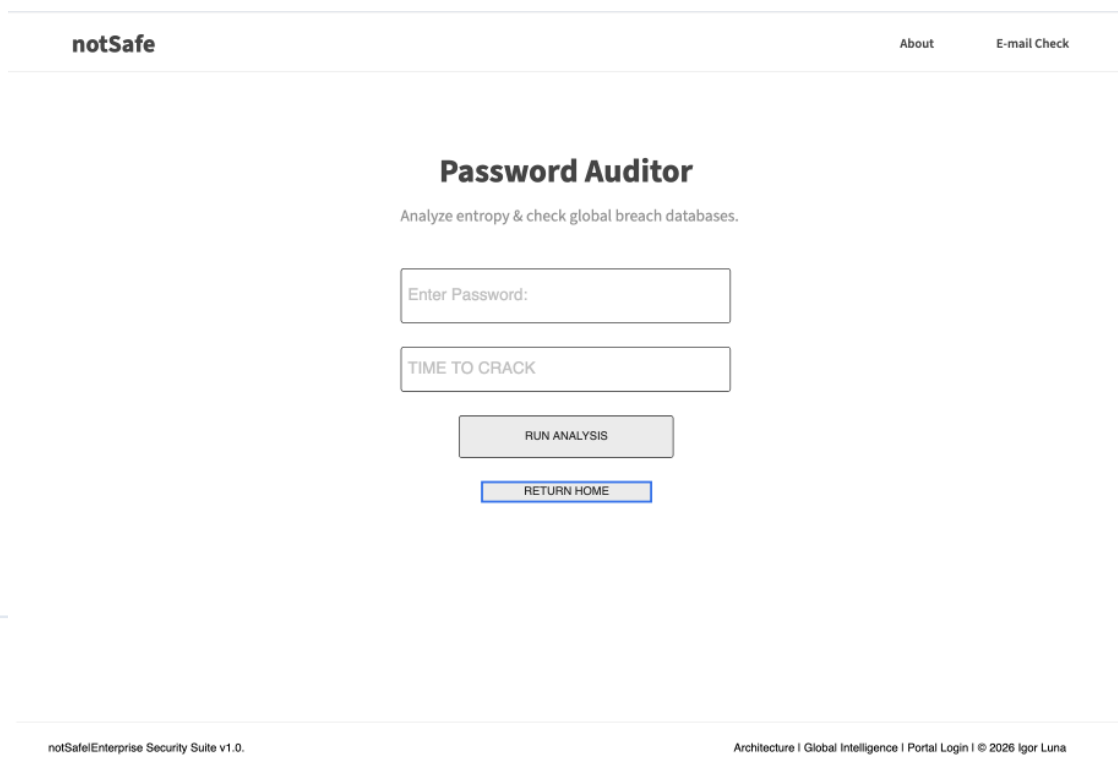


- **Les Wireframes (Squelette)** : J'ai ensuite précisé l'emplacement des éléments d'interface (boutons, inputs, graphiques). C'est ici que j'ai réfléchi à l'ergonomie, notamment pour que le passage du desktop au mobile reste intuitif.

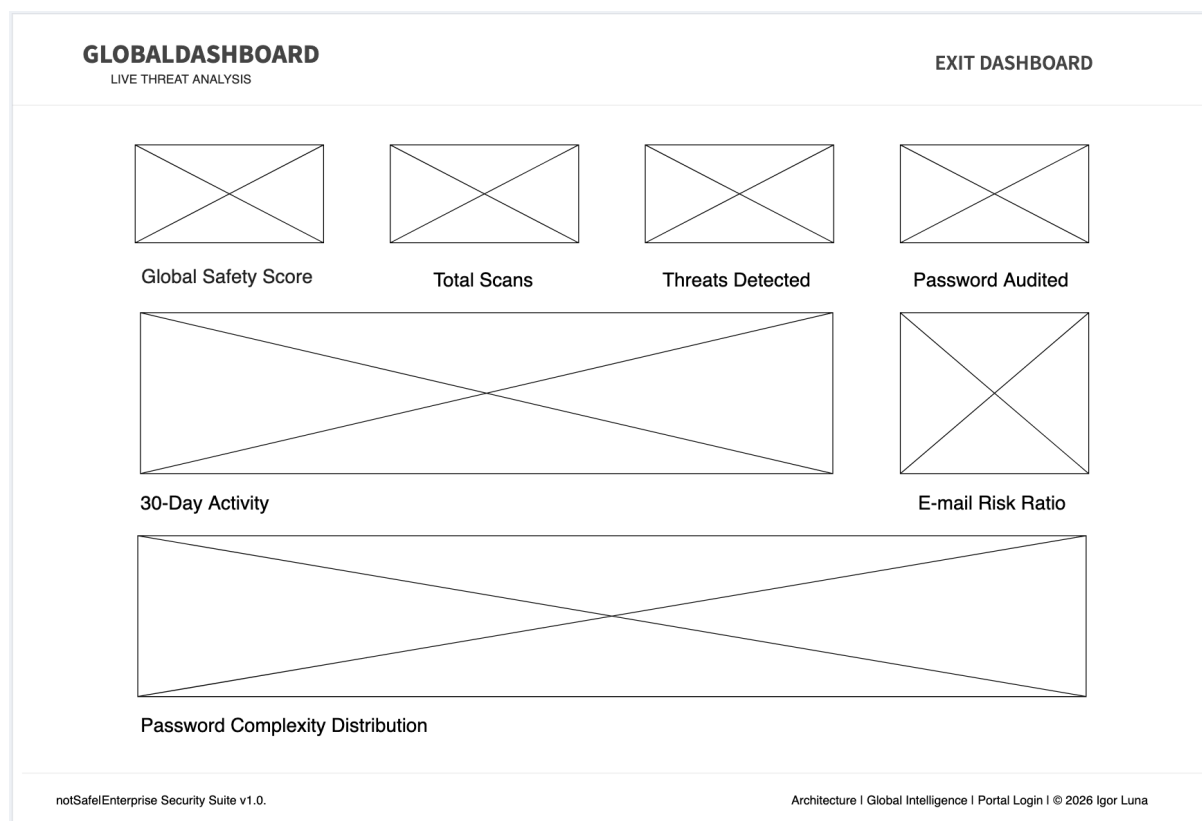
Wireframe de la Home Page - notSafe.



Wireframe Password Auditor - notSafe.



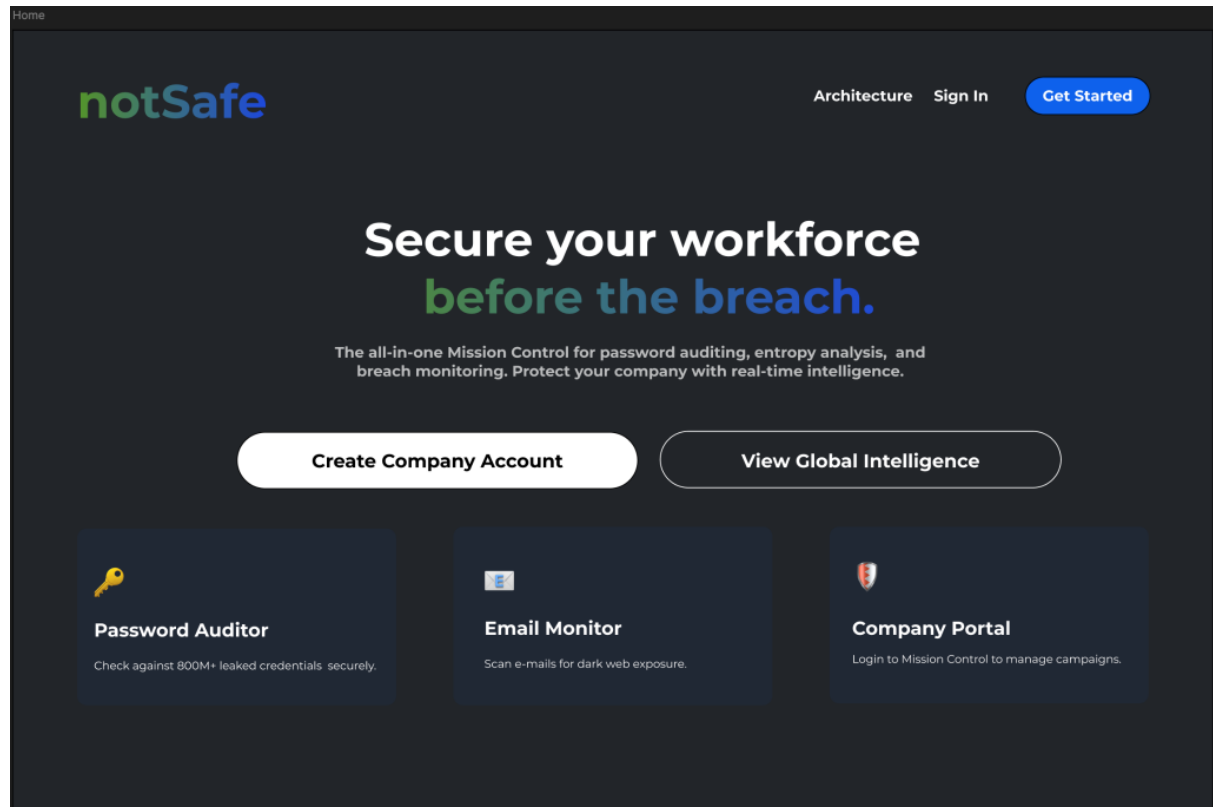
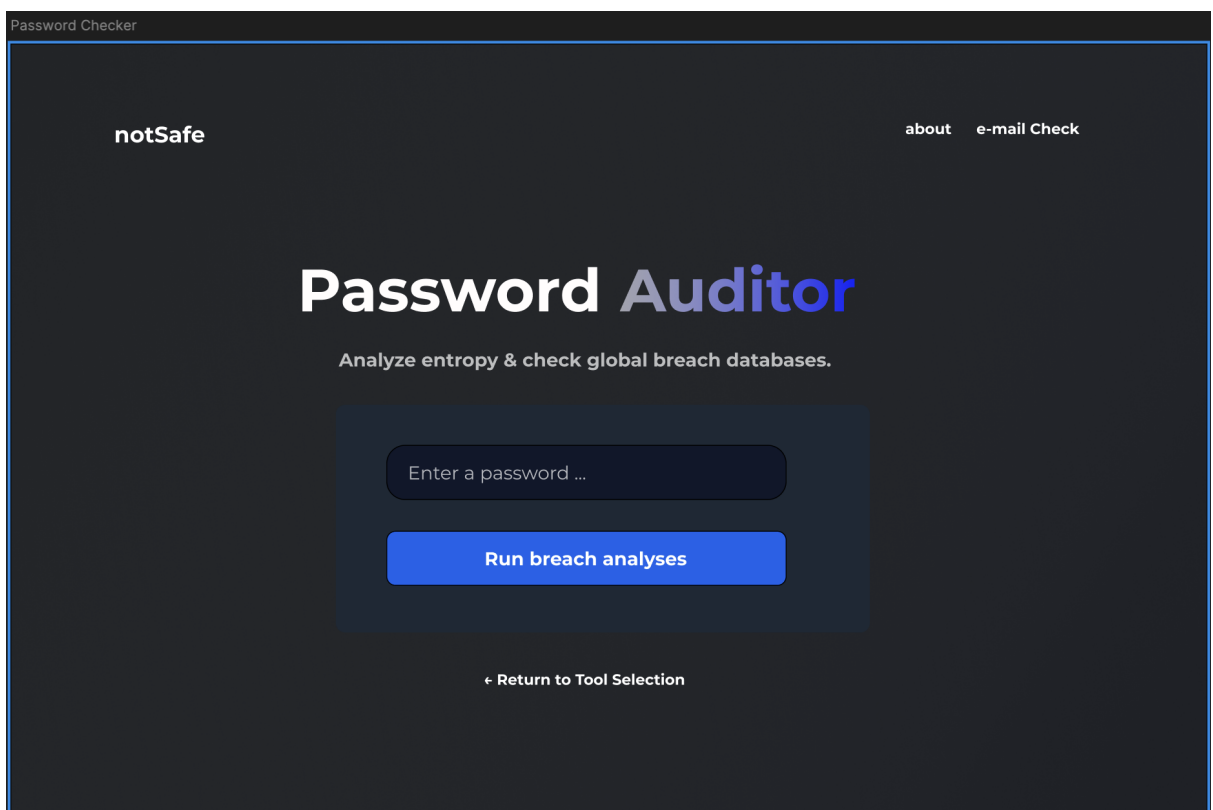
Wireframe Dashboard - notSafe.



4.3 Maquette Haute Fidélité et Identité Visuelle

La dernière étape a consisté à appliquer la charte graphique. J'ai choisi une esthétique moderne type "SaaS" avec des contrastes élevés pour améliorer l'accessibilité.

- **Choix graphiques :** Utilisation de composants réutilisables pour garantir une cohérence visuelle sur toute l'application.
- **Interactivité :** Les maquettes haute fidélité intègrent les états de survol (hover) et les retours visuels (couleurs d'entropie : rouge pour faible, vert pour robuste).

High Fidelity de la Home Page - notSafe.*High Fidelity Password Checker - notSafe.*

High Fidelity du Dashboard - notSafe.



4.4 Accessibilité et Responsive

En tant que **Développeur Web et Web Mobile**, il était essentiel que l'outil soit parfaitement utilisable sur smartphone. J'ai conçu les maquettes en pensant au "Mobile First", permettant à un employé de tester ses accès rapidement depuis son téléphone.

5. Architecture de données — Modélisation Merise

5.0 Note sur la méthodologie de modélisation

Plus proche de la culture britannique et brésilienne, l'apprentissage de la

méthode Merise a représenté un défi particulier — habitué à l'UML pour la modélisation, j'ai découvert une approche très structurée et spécifiquement ancrée dans la culture informatique française."

5.1 Architecture hybride SQL + NoSQL

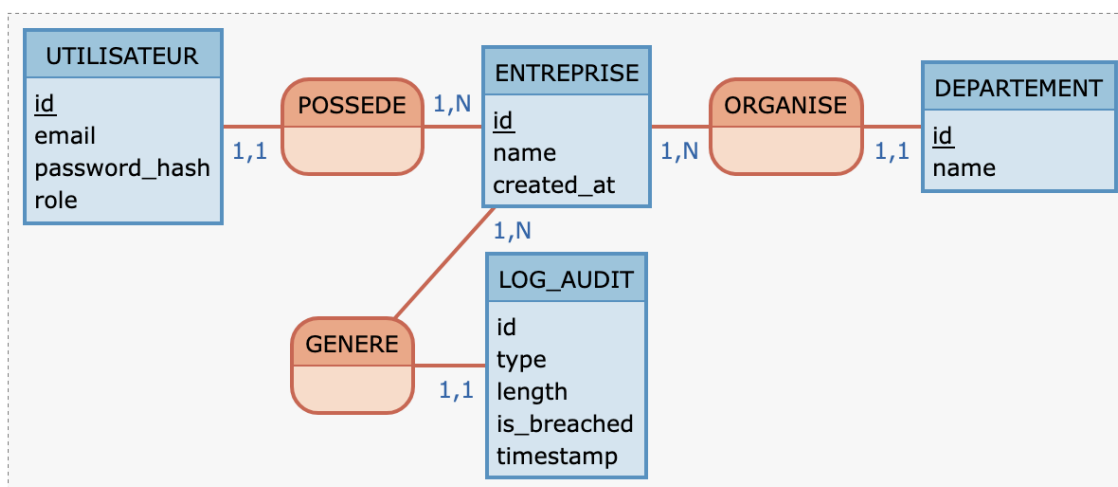
À l'origine, j'avais conçu l'application pour fonctionner exclusivement en **NoSQL (MongoDB)**. Pour démontrer ma maîtrise des bases de données relationnelles et assurer l'intégrité des comptes, j'ai fait évoluer l'architecture vers un modèle hybride :

- **Relationnel (PostgreSQL / SQLite) :** Gère les entités métier structurées (Entreprises, Utilisateurs, Départements) via l'ORM **SQLAlchemy**.
- **Non-Relationnel (MongoDB) :** Conservé spécifiquement pour les **logs anonymes** générés en volume à chaque audit, où la flexibilité du NoSQL est un atout majeur.

5.2 Modèle Conceptuel de Données (MCD)

La phase de conception a débuté par l'élaboration du **MCD** selon la méthode Merise. Ce schéma définit les règles de gestion métier :

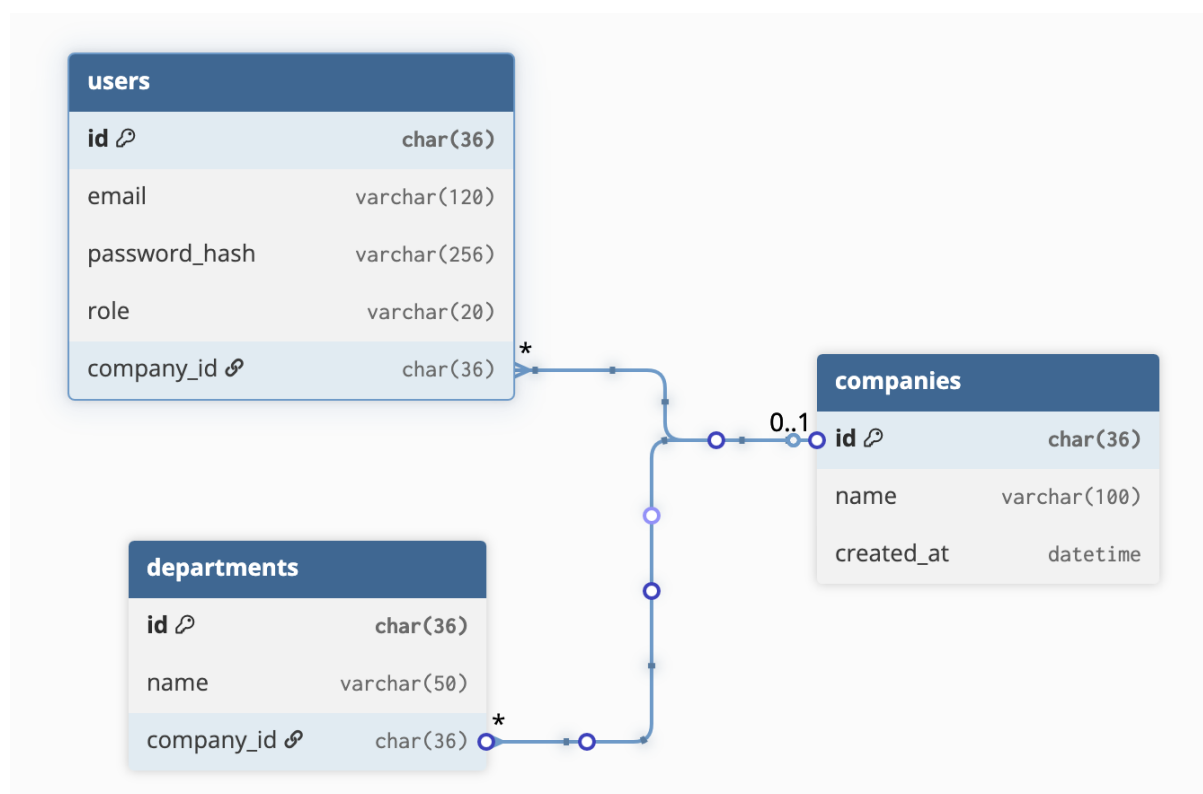
- Une **Entreprise** **POSSEDE** un ou plusieurs **Utilisateurs** (Admins).
- Une **Entreprise** **ORGANISE** son activité en plusieurs **Départements**.
- Un **Département** **GENERE** des **Logs d'Audit** lors des tests effectués par les employés.



- **DEPARTEMENT** (id, name, #id 2)
- **ENTREPRISE** (id, name, created_at)
- **LOG_AUDIT** (id 1, type, length, is_breached, timestamp, #id 2)
- **UTILISATEUR** (id, email, password_hash, role, #id 2)

5.3 Modèle Physique de Données (MPD)

Le MPD traduit ces logiques en types de données concrets, prêts à être implémentés dans le code.



5.4 Dictionnaire de données

Le dictionnaire de données de notSafe présente les deux bases de données utilisées dans le projet : une base SQL pour les données structurées comme les entreprises, les utilisateurs et les départements, et une collection MongoDB pour stocker les résultats d'audit de manière anonyme.

5.4.1 Stockage Relationnel (SQL - PostgreSQL)

Cette partie gère les données structurées du projet : les entreprises, leurs utilisateurs et leurs départements. Ces trois tables sont liées entre elles par des clés étrangères, ce qui garantit la cohérence des données.

SQL TABLES RELATIONNELLES

TABLE 01 Entreprises Entité cliente principale companies			
CHAMP	LIBELLÉ	TYPE	CONTRAINTES
id	Identifiant	Entier	PK NN
name	Nom	Texte	NN
created_at	Date de création	Date	AUTO

TABLE 02 Départements Unités organisationnelles departments			
CHAMP	LIBELLÉ	TYPE	CONTRAINTES
id	Identifiant	Entier	PK NN
name	Nom	Texte	NN
company_id	Réf. Entreprise	Entier	FK NN

TABLE 03 Utilisateurs Administrateurs & auditeurs users			
CHAMP	LIBELLÉ	TYPE	CONTRAINTES
id	Identifiant	Entier	PK NN
email	Adresse email	Texte	UNIQUE NN
password_hash	Mot de passe	Texte	NN
role	Rôle	Texte	DEFAULT
company_id	Réf. Entreprise	Entier	FK NN

5.4.2 Stockage Documentaire (NoSQL - MongoDB)

La collection `checks` enregistre un log à chaque fois qu'un employé effectue une vérification sur le portail. J'ai choisi MongoDB pour cette partie car les logs arrivent en grand volume et n'ont pas besoin d'être liés aux autres tables.

NoSQL COLLECTION MONGODB

COLLECTION 04

Logs d'audit

Données anonymes générées à chaque vérification

checks

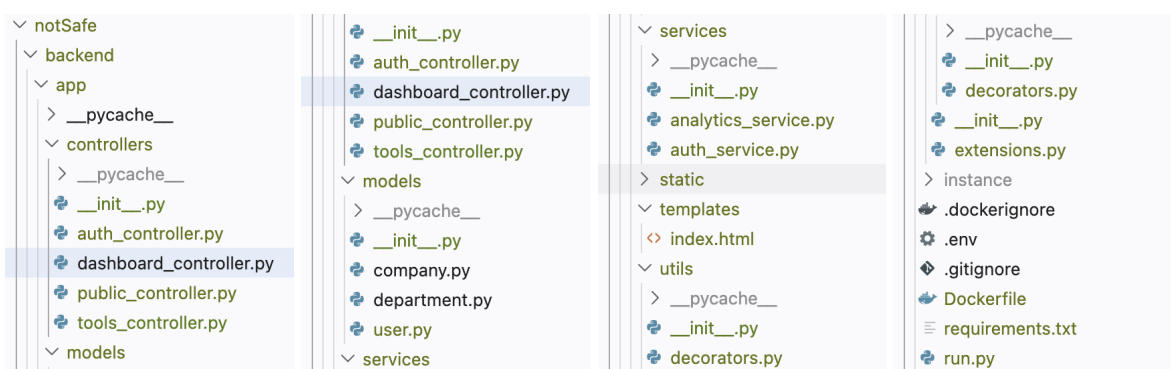
CHAMP	LIBELLÉ	TYPE	CONTRAINTES
<code>_id</code>	Identifiant	ObjectId	AUTO
<code>type</code>	Type d'audit	Texte	NN
<code>length</code>	Longueur	Entier	—
<code>is_breached</code>	Compromis	Booléen	NN
<code>company_id</code>	Réf. Entreprise	Texte	—
<code>department</code>	Département	Texte	—
<code>timestamp</code>	Date / Heure	Date	AUTO

6. Développement Backend — API Flask et sécurité

6.1 Flask - Le choix du framework

Pour le backend de notSafe, j'ai choisi Python, notamment pour approfondir mes compétences dans ce langage et travailler sur un projet concret de cette envergure — plutôt que de partir sur Node.js ou un autre langage backend. Cela m'a permis d'aller au-delà des exercices habituels, en confrontant mes compétences à des cas concrets : gestion des routes, structuration du code, manipulation de données et interaction avec une base de données.

Pour structurer cette API, j'ai opté pour Flask, un micro-framework plus simple et plus léger que Django. Flask m'a laissé construire mon API à ma façon — ce qui m'a permis de comprendre chaque brique que je mettais en place plutôt que de suivre un cadre automatisé. C'est un choix cohérent avec la formation : léger, lisible, et idéal pour construire une API REST sans surcharge inutile.



6.2 La structure de l'API

J'ai organisé le backend en couches séparées, où chaque dossier a une responsabilité précise.

La couche **models/** définit les trois entités SQL du projet. Chaque fichier correspond à une table : `company.py`, `user.py` et `department.py`. Ces modèles sont écrits avec SQLAlchemy, ce qui me permet de manipuler la base de données en Python sans écrire de SQL brut.

```
notSafe > backend > app > models > user.py > User
1  from app.extensions import db
2  import uuid
3  import jwt
4  import os
5
6  def generate_uuid():
7      return str(uuid.uuid4())
8
9  class User(db.Model):
10     __tablename__ = 'users'
11
12     id = db.Column(db.String(36), primary_key=True, default=generate_uuid)
13     email = db.Column(db.String(120), unique=True, nullable=False)
14     password_hash = db.Column(db.String(256), nullable=False)
15     role = db.Column(db.String(20), default='admin')
16
17     # Foreign Key
18     company_id = db.Column(db.String(36), db.ForeignKey('companies.id'), nullable=False)
19
20     # --- THIS WAS MISSING ---
21     @staticmethod
22     def verify_token(token):
23         try:
24             payload = jwt.decode(token, os.getenv('SECRET_KEY'), algorithms=["HS256"])
25             return payload['user_id']
26         except:
27             return None
```

La couche **controllers/** contient quatre fichiers qui correspondent aux quatre grandes fonctionnalités de l'application: `auth_controller.py` gère l'inscription et la connexion, `dashboard_controller.py` gère tout ce qui concerne l'espace admin, `tools_controller.py` gère les vérifications de mots de passe et les logs, et `public_controller.py` expose les données accessibles sans authentification. Chaque controller reçoit la requête HTTP, vérifie que les données sont valides, puis délègue le traitement au service correspondant.

```
notSafe > backend > app > controllers > auth_controller.py > ...
9  class RegisterController(Resource):
10     def post(self):
11         try:
12             company_id = AuthService.register_company(
13                 data.get('name'),
14                 data.get('email'),
15                 data.get('password')
16             )
17             return {"message": "Created", "company_id": company_id}, 201
18         except ValueError as e:
19             return {"error": str(e)}, 400
20
21 @ns.route('/login')
22 class LoginController(Resource):
23     def post(self):
24         """Login and get Token"""
25         data = request.get_json()
26         result = AuthService.login(data.get('email'), data.get('password'))
27         if result:
28             return result, 200
29         return {"error": "Invalid credentials"}, 401
30
31
32
33
```

La couche **services/** contient la logique métier, complètement détachée du HTTP. `auth_service.py` gère la création des comptes, la vérification des mots de passe et la génération des tokens JWT. `analytics_service.py` calcule les statistiques du dashboard à partir des données MongoDB.

```
39 # — LOGIN —
40 @staticmethod
41 def login(email, password):
42     user = User.query.filter_by(email=email).first()
43     if user and bcrypt.check_password_hash(user.password_hash, password):
44         token = jwt.encode(
45             {
46                 'user_id': user.id,
47                 'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=24)
48             },
49             os.getenv('SECRET_KEY'),
50             algorithm="HS256"
51         )
52         return {"token": token, "role": user.role, "company_id": user.company_id}
53     return None
54
```

Enfin, `decorators.py` contient le middleware `@token_required` — une fonction réutilisable qui vérifie la validité du token JWT avant de laisser passer la requête. Il suffit d'ajouter ce décorateur au-dessus d'une route pour la protéger.

notSafe > backend > app > utils >  `decorators.py` >  `token_required`

```
1  from functools import wraps
2  from flask import request, current_app
3  import jwt
4  from app.models.user import User
5  from app.extensions import db
6
7  def token_required(f):
8      @wraps(f)
9      def decorated(*args, **kwargs):
10         token = None
11
12         # 1. Get Token from Header
13         if 'Authorization' in request.headers:
14             auth_header = request.headers['Authorization']
15             if auth_header.startswith('Bearer '):
16                 token = auth_header.split(" ")[1]
17
18         if not token:
19             return {'message': 'Token is missing!'}, 401
20
21         try:
22             # 2. Decode Token
23             data = jwt.decode(token, current_app.config['SECRET_KEY'], algorithms=["HS256"])
24
25             # 3. Find User — use db.session.get() (SQLAlchemy 2.x compatible)
26             current_user = db.session.get(User, data['user_id'])
27             if not current_user:
28                 return {'message': 'User not found'}, 401
29
30         except jwt.ExpiredSignatureError:
31             return {'message': 'Token has expired!'}, 401
32         except jwt.InvalidTokenError as e:
33             return {'message': 'Token is invalid!', 'error': str(e)}, 401
34         except Exception as e:
35             return {'message': 'Authentication error', 'error': str(e)}, 401
36
37         # 4. Pass current_user to the protected route
38         return f(*args, current_user=current_user, **kwargs)
39
40     return decorated
```

6.3 Connexion à MongoDB

En parallèle de la base SQL, j'ai connecté MongoDB via Flask-PyMongo pour stocker les logs d'audit anonymes. Chaque vérification effectuée sur le portail génère un document dans la collection `checks` contenant le type d'audit, la longueur du mot de passe, le résultat de la vérification et un horodatage.

J'ai choisi MongoDB pour cette partie pour une raison concrète : les logs arrivent en volume important et n'ont pas besoin d'être liés aux autres tables SQL. De plus, un log de vérification de mot de passe n'a pas exactement la même structure qu'un log de scan email — MongoDB accepte cette variation naturellement, là où une base relationnelle m'aurait obligé à créer des colonnes vides ou des tables supplémentaires.

La connexion est initialisée dans `extensions.py` au démarrage de l'application Flask, ce qui la rend disponible dans tous les controllers sans avoir à la recréer à chaque requête. Les insertions se font en une seule ligne, et les statistiques du dashboard sont calculées via des pipelines d'agrégation — l'équivalent MongoDB d'une requête SQL avec GROUP BY.

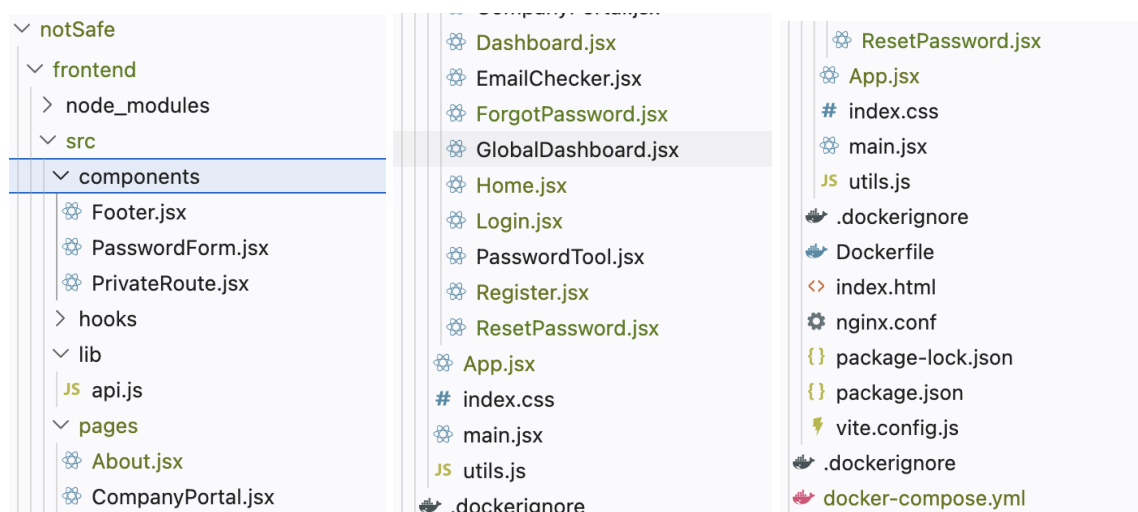
```
153 @ns.route('/log-dept-check')
154 class LogDeptCheck(Resource):
155     def post(self):
156         """Log an anonymous department password check"""
157         data = request.get_json() or {}
158         try:
159             mongo.db.checks.insert_one({
160                 "type": "dept_check",
161                 "company_id": data.get('company_id', ''),
162                 "department": data.get('department', ''),
163                 "length": data.get('length', 0),
164                 "is_breached": data.get('is_breached', False),
165                 "timestamp": datetime.datetime.utcnow()
166             })
167         except Exception:
168             pass
169         return {"status": "logged"}, 201
```

7. Développement Frontend — React

7.1 Architecture des composants

L'application React est organisée par fonctionnalités, chaque dossier ayant une responsabilité précise. Les pages se trouvent dans `pages/`, les composants réutilisables dans `components/`, et la logique partagée dans `hooks/`. L'instance

Axios centralisée dans `lib/api.js` gère automatiquement l'injection du token et les redirections en cas d'expiration.



7.2 Les pages principales

L'application compte 11 pages. La page d'accueil `Home` présente les trois outils publics disponibles sans connexion. `PasswordTool` permet l'audit de mot de passe avec calcul d'entropie en temps réel et vérification HIBP. `EmailChecker` lance un scan d'exposition via XposedOrNot. `CompanyPortal` est le portail employé accessible via un lien unique par entreprise — chaque employé sélectionne son département et teste son mot de passe de manière complètement anonyme. `Dashboard` est l'espace admin protégé avec les métriques de risque, le graphique par département et la gestion des accès. `GlobalDashboard` affiche les statistiques publiques globales sur 30 jours sous forme de graphiques. Enfin, `Login`, `Register`, `ForgotPassword` et `ResetPassword` couvrent l'ensemble du flow d'authentification, du premier accès jusqu'à la réinitialisation du mot de passe.

7.3 Gestion des erreurs et des états

La gestion des erreurs est centralisée dans `lib/api.js` via un intercepteur Axios. Si le serveur retourne une erreur 401 — token expiré ou invalide — l'utilisateur est automatiquement déconnecté et redirigé vers la page de connexion, sans avoir à gérer ce cas dans chaque composant individuellement.

Dans les formulaires, chaque action asynchrone suit le même schéma : un état `loading` désactive le bouton pendant la requête pour éviter les doubles soumissions, et un état `error` affiche un message précis si la requête échoue. Ce comportement est visible sur les pages Login, Register, ForgotPassword et ResetPassword.

notSafe > frontend > src > lib > JS api.js > ...

```
1 import axios from 'axios';
2
3 // 1. Create a centralized Axios instance
4 const api = axios.create({
5   baseURL: '/api/v1', // This matches your Nginx/Vite proxy
6   headers: {
7     'Content-Type': 'application/json',
8   },
9 });
10
11 // 2. Request Interceptor: Auto-injects the Token
12 api.interceptors.request.use((config) => {
13   const token = localStorage.getItem('token');
14   if (token) {
15     config.headers.Authorization = `Bearer ${token}`;
16   }
17   return config;
18 });
19
20 // 3. Response Interceptor: Auto-logout on 401
21 api.interceptors.response.use(
22   (response) => response,
23   (error) => {
24     if (error.response && error.response.status === 401) {
25       // Token expired or invalid -> Clear storage and redirect
26       localStorage.removeItem('token');
27       localStorage.removeItem('role');
28       window.location.href = '/login';
29     }
30     return Promise.reject(error);
31   }
32 );
33
34 export default api;
```

```

export default function Login() {
  const [formData, setFormData] = useState({ email: '', password: '' });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      const res = await api.post('/auth/login', formData);
      localStorage.setItem('token', res.data.token);
      localStorage.setItem('role', res.data.role);
      navigate('/dashboard');
    } catch (err) {
      console.error(err);
      setError(err.response?.data?.error || 'Connection failed.');
```

```

    } finally {
      setLoading(false);
    }
  };

  <button
    type="submit"
    className="btn btn-danger w-100 py-2 fw-bold shadow"
    disabled={loading}
  >
    {loading ? 'AUTHENTICATING...' : 'ACCESS DASHBOARD'}
  </button>

```

8. Securite

La sécurité de notSafe repose sur quatre mécanismes mis en place dès la conception du projet, et non ajoutés en fin de développement.

8.1 K-Anonymité (HIBP)

La vérification de mot de passe ne transmet jamais le mot de passe complet à un service externe. Côté navigateur, le mot de passe est haché en SHA-1 via l'API Web Crypto — une API native du navigateur, sans dépendance externe.

Seuls les 5 premiers caractères hexadécimaux du hash, appelés le préfixe, sont envoyés au backend Flask, qui les transmet à l'API HIBP. En retour, HIBP renvoie une liste de suffixes correspondants. La comparaison finale se fait entièrement côté client. Le mot de passe complet ne quitte jamais le navigateur.

notSafe > frontend > src > hooks > JS usePasswordHasher.js > ...

```

1  import { useState } from 'react';
2
3  export const usePasswordHasher = () => {
4    const [isLoading, setIsLoading] = useState(false);
5    const [error, setError] = useState(null);
6
7    const hashPassword = async (password) => {

```

8.2 JWT — JSON Web Token

L'authentification dans notSafe repose sur un système de token. Une fois que l'utilisateur se connecte avec succès, le serveur Flask crée un JWT. Ce JWT est une preuve d'identité signée qui contient les informations de l'utilisateur (son ID, son rôle) et une date de fin de validité fixée à 24 heures. Ce token est stocké côté client dans le localStorage et est automatiquement envoyé avec toutes les requêtes HTTP grâce à un intercepteur Axios.

Côté serveur, le décorateur `@token_required` agit comme un firewall : il vérifie que le token est bien valide avant de laisser passer la requête vers le contrôleur concerné. Ce décorateur est détaillé dans ce dossier de projet (dans `decorators.py`) : c'est un mécanisme réutilisable qui contrôle la validité du token JWT. Il suffit de l'ajouter au-dessus d'une fonction (route) pour la sécuriser. Si le token est manquant, expiré ou si ses données sont fausses, la requête est

immédiatement rejetée avec une erreur 401 (Accès Interdit), sans jamais atteindre la logique métier.

Pour le processus de réinitialisation de mot de passe, nous générons un token spécial. Celui-ci a une durée de vie réduite à 1 heure et porte une information spécifique (type: password_reset), ce qui garantit qu'il ne peut être utilisé que pour la réinitialisation et non pour se connecter comme un utilisateur normal.

[veuillez consulter la capture d'écran à la page 23 pour voir decorator.py]

```
# — PASSWORD RESET: REQUEST —————
@staticmethod
def create_reset_token(email: str):
    """
    Generate a short-lived (1 h) signed reset token for the given email.
    Returns the token string, or None if the email does not exist.

    The token is a JWT containing:
    - sub : user.id
    - email : user.email          (so we can verify it hasn't changed)
    - type : "password_reset"     (so it can't be used as a login token)
    - jti : random nonce         (one-time use guard – store in DB if needed)
    - exp : now + 1 hour
    """
    user = User.query.filter_by(email=email).first()
    if not user:
        return None          # don't reveal whether the email exists

    token = jwt.encode(
        {
            'sub': user.id,
            'email': user.email,
            'type': 'password_reset',
            'jti': secrets.token_hex(16), # unique nonce
            'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
        },
        os.getenv('SECRET_KEY'),
        algorithm="HS256"
    )
    return token
```

8.3 Bcrypt — Hachage des mots de passe

Aucun mot de passe n'est jamais stocké en clair dans la base de données. À l'inscription, le mot de passe est transformé par l'algorithme Bcrypt avant d'être enregistré.

Bcrypt présente deux caractéristiques importantes. D'abord, il génère

automatiquement un salt unique à chaque hachage — une valeur aléatoire ajoutée au mot de passe avant transformation, ce qui fait que deux utilisateurs avec le même mot de passe auront des hashes complètement différents en base.

Ensuite, Bcrypt est délibérément lent, environ 250ms par vérification. Ce délai est imperceptible pour un utilisateur qui se connecte normalement, mais il rend une attaque par force brute extrêmement coûteuse en temps et en ressources.

```
class AuthService:
    """Handles all Authentication Logic."""

    # — REGISTER —————
    @staticmethod
    def register_company(name, email, password):
        if User.query.filter_by(email=email).first():
            raise ValueError("User already exists")

        new_company = Company(name=name)
        db.session.add(new_company)
        db.session.flush()

        for dept_name in DEFAULT_DEPARTMENTS:
            dept = Department(name=dept_name, company_id=new_company.id)
            db.session.add(dept)

        pw_hash = bcrypt.generate_password_hash(password).decode('utf-8')
        new_user = User(email=email, password_hash=pw_hash, company_id=new_company.id)
        db.session.add(new_user)
        db.session.commit()
        return new_company.id

    # — LOGIN —————
    @staticmethod
    def login(email, password):
        user = User.query.filter_by(email=email).first()
        if user and bcrypt.check_password_hash(user.password_hash, password):
            token = jwt.encode(
                {
                    'user_id': user.id,
                    'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=24)
                },
                os.getenv('SECRET_KEY'),
                algorithm="HS256"
            )
            return {"token": token, "role": user.role, "company_id": user.company_id}
        return None
```

8.4 Validation des données et protection contre les injections

Tous les noms de départements saisis par l'utilisateur sont traités en deux étapes avant insertion en base. D'abord, les balises HTML sont supprimées pour neutraliser les tentatives d'injection de code. Ensuite, la valeur restante est validée par une expression régulière qui n'autorise que les caractères attendus — lettres, chiffres, espaces, tirets et apostrophes. Toute valeur qui ne correspond pas à ce format est rejetée.

Concernant les injections SQL, l'utilisation de SQLAlchemy comme ORM les rend impossibles par construction — les requêtes sont préparées automatiquement, sans jamais concaténer des valeurs utilisateur dans du SQL brut.

```
__
_TAG_RE = re.compile(r'<[>]+>')

def sanitize_name(value: str) -> str:
    """Strip HTML/script tags then HTML-escape any remaining special chars."""
    stripped = _TAG_RE.sub('', value)
    return html.escape(stripped).strip()

125
126     if not name:
127         return {"error": "Department name required (no HTML/scripts allowed)"}, 400
128     if len(name) > 50:
129         return {"error": "Department name too long (max 50 chars)"}, 400
130     # Whitelist: letters, numbers, spaces, hyphens, ampersands, apostrophes
131     if not re.match(r"^[w\s\-\&]+'$", name):
132         return {"error": "Department name contains invalid characters"}, 400
133
```

9. Déploiement — Docker

9.1 Pourquoi Docker ?

Pendant ma formation, j'ai souvent rencontré le problème suivant : mon code fonctionnait sur mon ordinateur mais pas sur celui d'un autre. Docker résout ça. Il permet d'emballer l'application et toutes ses dépendances dans des conteneurs standardisés. N'importe qui peut lancer le projet complet avec une seule commande :

```
docker-compose up -d
```

9.2 Architecture des conteneurs

Le fichier `docker-compose.yml` orchestre trois services indépendants qui communiquent entre eux sur un réseau interne :

1. `mongo` — Base de données NoSQL (MongoDB) pour le stockage flexible des données. Elle utilise un volume persistant (`mongo_data`) pour garantir que les données persistent même après redémarrage des conteneurs.
2. `backend` — Service d'API Python/Flask. Il gère la logique métier et l'authentification. Son fonctionnement dépend de la connexion établie avec le service `mongo`.
3. `frontend` — Service qui sert les fichiers statiques compilés de l'application React. Dans ce contexte, il agit comme le point d'entrée public de l'application.

```
notSafe > docker-compose.yml
1  version: '3.8'
2
3  services:
4
5      # 1. NoSQL Database
6      mongo:
7          image: mongo:latest
8          container_name: notsafe-mongo
9          ports:
10             - "27017:27017"
11          volumes:
12             - mongo_data:/data/db
13
14      # 2. Backend (Python/Flask)
15      backend:
16          build: ./backend
17          container_name: notsafe-backend
18          ports:
19             - "5001:5001"
20
21      environment:
22          - MONGO_URI=mongodb://mongo:27017/notsafe_db
23          - SECRET_KEY=dev_key_only
24          - FLASK_APP=run.py
25          - FLASK_DEBUG=1
26      depends_on:
27          - mongo
28      volumes:
29          - ./backend:/app
30
31      # 3. Frontend (React built, served by nginx)
32      frontend:
33          build: ./frontend
34          container_name: notsafe-frontend
35          ports:
36             - "5173:80" # nginx listens on 80 internally
37          depends_on:
38             - backend
```

9.3 Gestion des secrets

Aucune information sensible n'est présente dans le code source. Toutes les clés et mots de passe sont injectés au démarrage via un fichier `.env` qui n'est jamais versionné — il est listé dans `.gitignore`, ce qui garantit qu'il ne sera jamais publié sur GitHub par accident.

- `SECRET_KEY` — Clé de signature des JWT (générée aléatoirement).
- `DATABASE_URL` — URL de connexion PostgreSQL.
- `MONGO_URI` — URI de connexion MongoDB.

10. Bilan des compétences DWWM

Ce tableau met en correspondance chaque compétence du référentiel DWWM avec sa réalisation concrète dans le projet notSafe. Chaque ligne correspond à quelque chose que j'ai réellement construit et que je suis en mesure d'expliquer et de démontrer lors de la soutenance.

Compétence DWWM (Référentiel)	Réalisation concrète dans notSafe
Maquetter une application	Conception complète sur Figma : zoning → wireframes → maquettes haute fidélité desktop et mobile.
Réaliser une interface utilisateur responsive	11 pages React avec Bootstrap 5, glassmorphism CSS, composants réutilisables et adaptation mobile.
Développer une interface utilisateur dynamique	Calcul d'entropie temps réel (useEffect), graphiques Chart.js (Bar, Line, Doughnut), mise à jour sans rechargement de page.
Créer une base de données	Modélisation Merise complète (MCD → MLD → MPD), implémentation SQLAlchemy, architecture hybride SQL + MongoDB.
Développer les composants d'accès aux données	ORM SQLAlchemy pour SQL + PyMongo pour MongoDB, pipelines d'agrégation MongoDB pour les analytics.
Développer une API REST	9 endpoints Flask avec validation des entrées, codes HTTP appropriés (200/201/400/401/404/409/502) et rate limiting.
Mettre en place une authentification	JWT avec @token_required, Bcrypt, flow reset mot de passe complet, anti-énumération, intercepteur Axios 401.
Gérer un projet Agile	7 Sprints Scrum de 2 semaines, backlog Git, branches par fonctionnalité, tests Postman sur chaque endpoint.
Déployer une application	Conteneurisation Docker complète, docker-compose multi-services, gestion des secrets via .env, build Vite pour la production.

11. Conclusion personnelle

11.1 Les difficultés rencontrées

Ce projet n'a pas été linéaire. À l'origine, notSafe devait être un simple vérificateur de mots de passe — une seule page, une seule fonctionnalité. Au fil du développement, l'idée a grandi : ajout du scan email, puis du portail employé, puis du dashboard multi-entreprise. L'architecture a évolué en même temps.

J'avais commencé avec une base NoSQL unique, avant de réaliser que les données structurées comme les entreprises et les utilisateurs avaient besoin des contraintes d'une base relationnelle. C'est ce qui m'a conduit à l'architecture hybride SQL + MongoDB que j'ai finalement mise en place. Plusieurs blocages m'ont appris autant que les parties qui ont fonctionné du premier coup.

Le manque de planification initiale — Mon plus grand blocage n'était pas technique. Au départ, je me suis lancé dans le code sans avoir une idée claire et stable de ce que je voulais construire. J'ajoutais des fonctionnalités, j'en supprimais d'autres, je changeais de direction en cours de route. Ça m'a coûté du temps et quelques refactorisations complètes. C'est ce qui m'a convaincu de l'utilité réelle de la phase de conception — le Sprint 0 et la modélisation Merise ne sont pas des formalités, ce sont des outils qui évitent de coder deux fois la même chose.

La validation MongoDB — J'ai identifié que le endpoint `/log-dept-check` acceptait n'importe quel `company_id` sans vérifier qu'il existait en base SQL. Concrètement, cela aurait pu créer des logs rattachés à des entreprises inexistantes. J'ai documenté ce point comme axe d'amélioration — c'est une limite que je connais et que je saurais corriger en production.

Register.jsx — En revoyant mon code, j'ai remarqué que cette page utilisait `fetch()` natif alors que toutes les autres utilisaient mon instance Axios centralisée `api.js`. Une petite erreur d'inattention, mais qui montre l'importance de relire son propre code avec du recul.

11.2 Ce que je retiens

Avant cette formation, des termes comme API, JWT ou Docker me semblaient appartenir à un autre monde. Aujourd'hui, je les ai non seulement utilisés, mais j'ai compris pourquoi ils existent et comment ils s'articulent entre eux.

notSafe est un MVP fonctionnel, documenté, versionné et déployable. Il couvre l'ensemble du cycle d'une application web : une API sécurisée, deux bases de données, une interface utilisateur complète, des tests et une conteneurisation Docker. Des fonctionnalités comme l'envoi d'emails réel, l'authentification à deux facteurs ou les tests de charge ne sont pas encore intégrées, mais la base est là pour les accueillir.

Ce que je retiens surtout, c'est qu'un projet ne se résume pas à du code qui fonctionne. Il se documente, se versionne, se structure pour que quelqu'un d'autre puisse le reprendre. C'est ce que j'ai essayé de faire avec notSafe, et c'est ce que je veux continuer à faire.

12. Annexes

Annexe A — Tests unitaires

Le projet inclut une suite de tests organisée en 4 fichiers selon le type de test :

- `test_unit_pure.py` — 32 tests unitaires sur les fonctions pures (`sanitize_name`, validation des préfixes HIBP, validation email, whitelist département). Zéro dépendance, zéro mock, exécutable en 0.001 seconde.
- `test_integration_endpoints.py` — Tests d'intégration HTTP avec Flask test client + SQLite `:memory:` + MongoDB mocké.
- `test_functional_journeys.py` — Scénarios utilisateur complets (inscription → connexion → dashboard, flow reset mot de passe, portail employé).
- `test_security.py` — Tests d'attaque : JWT forgé, token expiré, attaque `alg:none`, XSS, tampering de token, isolation inter-entreprises.

Annexe B — Glossaire

Terme	Définition simple
API REST	Interface qui permet à deux logiciels de communiquer via des requêtes HTTP (comme un serveur de restaurant entre la salle et la cuisine).
Bcrypt	Algorithme qui transforme un mot de passe en une suite de caractères illisibles. Intentionnellement lent pour rendre les attaques non rentables.
Docker	Outil qui emballe une application dans un « conteneur » standardisé. Lance sur n'importe quel ordinateur sans problème de compatibilité.
HIBP	Have I Been Pwned — base de données mondiale des comptes piratés. notSafe l'interroge via un proxy pour ne jamais exposer les données des utilisateurs.
JWT	Badge numérique temporaire généré après connexion. Contient l'identité de l'utilisateur et une date d'expiration. Signé cryptographiquement.
K-Anonymité	Protocole qui permet de vérifier si une donnée est compromise sans jamais l'envoyer en entier à un service tiers.
ORM	Outil qui permet de manipuler une base de données en écrivant du Python plutôt que du SQL brut (SQLAlchemy dans ce projet).
Salt	Valeur aléatoire unique ajoutée au mot de passe avant hachage. Rend inutiles les attaques par dictionnaire précalculé (rainbow tables).
XSS	Attaque qui consiste à injecter du JavaScript malveillant dans une page web pour voler des données d'autres utilisateurs.

Annexe C — Veille technologique

Dans le cadre du projet et de ma formation, j'ai maintenu une veille régulière sur les sources suivantes :

- OWASP (owasp.org) — Référentiel des 10 vulnérabilités web les plus critiques. Consulté pour valider chaque choix de sécurité.
- ANSSI (ssi.gouv.fr) — Recommandations françaises sur les politiques de mots de passe et la sécurité des applications web.
- Troy Hunt / HaveIBeenPwned — Blog de l'auteur d'HIBP. Suivi des nouvelles fuites de données annoncées et des évolutions de l'API.
- CVE / NVD (nvd.nist.gov) — Surveillance des vulnérabilités connues dans les dépendances du projet (Flask, React, PyJWT...).

- The Hacker News / Krebs on Security — Actualité cybersécurité pour maintenir une vision du contexte des menaces actuelles.

Igor Luna de Oliveira — DWWM Session Avril 2026 — IMIE